

Recommandation de Stack Technologique pour Ko'Ride

Contexte et enjeux du projet Ko'Ride

Ko'Ride est une future application de covoiturage ciblant le marché indonésien. L'équipe de développement est junior (4 membres) avec des compétences variées : David (backend débutant/intermédiaire), Jean-Charles (fullstack junior), Yann (frontend Angular) et Léo (UX/UI). Le contexte impose plusieurs contraintes : la solution doit être performante, rapide à développer, à faible coût, et utilisable par des développeurs locaux en Indonésie. De plus, le backend devra être centralisé pour alimenter à la fois une application mobile et un site web (via une API REST ou GraphQL). La stack technologique choisie doit non seulement permettre de construire un MVP rapidement, mais aussi pouvoir évoluer vers une application en production à grande échelle.

L'équipe a déjà des préférences initiales : - **Frontend web** : Angular (cadre maîtrisé par Yann). - **Backend** : Java Spring Boot, ou une solution Node.js (Express ou NestJS en TypeScript). - **Mobile** : Pas d'expérience mobile spécifique à ce stade, mais ouverture à React Native, Flutter ou autres solutions cross-plateformes accessibles.

Il s'agit donc de proposer une architecture technique réaliste et progressive, tirant parti des bonnes pratiques observées chez les leaders internationaux du covoiturage (BlaBlaCar, Grab, Lyft, Uber) tout en l'adaptant à une petite équipe et au contexte local indonésien.

Leçons des stacks utilisées par BlaBlaCar, Grab, Lyft, Uber

Les grandes applications de mobilité partagée ont fait évoluer leur stack au fil du temps pour atteindre performance et scalabilité. Il est utile d'en tirer quelques enseignements avant de choisir la stack de Ko'Ride :

- **Architecture monolithique vs microservices** : Beaucoup de ces entreprises ont démarré avec un système monolithique simple, puis sont passées à une architecture microservices à mesure de leur croissance. Par exemple, Uber a débuté avec un backend monolithique écrit en Python et une base de données PostgreSQL unique ¹. Ce choix a suffi pendant la phase initiale, jusqu'à ce que la montée en charge et l'élargissement de l'équipe de développement rendent nécessaire une transition vers des microservices (afin de mieux segmenter les fonctionnalités et d'améliorer l'échelle) ¹ ². De même, BlaBlaCar a adopté plus tard une architecture à microservices pour gagner en scalabilité et permettre le déploiement indépendant des fonctionnalités ³. Ces approches montrent qu'un **monolithe bien conçu peut supporter un MVP et les débuts**, et que la **migration vers des microservices** peut venir ensuite lorsque les besoins l'imposent ⁴. Il est important de noter que passer trop tôt aux microservices ajoute de la complexité inutile pour une petite équipe – la sagesse conventionnelle recommande de commencer monolithique puis d'évoluer une fois la complexité avérée ⁴.
- **Langages et frameworks backend utilisés** : Les stacks des géants sont souvent polyglottes. Uber, par exemple, a au fil du temps exploité **plusieurs langages backend** au service de ses

microservices : Python (Flask) historiquement, Node.js pour son moteur de dispatch initial, puis Go et Java pour divers services ⁵ ⁶ . Uber indique que son moteur de calcul des trajets avait été écrit en Node.js à l'origine pour profiter de son modèle asynchrone et de sa capacité à gérer un grand nombre de connexions concurrentes, avant d'introduire Go pour des raisons de performance et de sécurité de typage ⁶ . Lyft a adopté principalement une architecture microservices en Python (Flask) côté serveur, avec des frontends web en AngularJS puis React selon les services ⁷ . Grab, de son côté, a fait évoluer son stack vers des microservices en Go (via leur framework interne Grab-Kit) pour tirer parti de la concurrence et de l'efficacité de Go ⁸ . **BlaBlaCar** a utilisé une stack mixte incluant notamment Node.js et Go, et s'est tourné vers Google Cloud Platform pour bénéficier d'une infrastructure scalable capable de répondre en temps réel aux besoins de sa communauté mondiale ⁹ . Ces exemples suggèrent qu'il n'y a pas un seul "langage magique" : les choix dépendent de l'étape de développement et des expertises disponibles. Pour Ko'Ride, cela signifie qu'il faut choisir un backend que l'équipe maîtrise et qui permette de prototyper rapidement, tout en étant suffisamment performant pour évoluer.

- **Stacks front-end et mobile des grands acteurs** : Les leaders ont mis l'accent sur l'expérience utilisateur native. Uber et Lyft ont développé des applications mobiles natives (Swift/Objective-C pour iOS, Java/Kotlin pour Android) afin d'optimiser les performances et l'intégration profonde (GPS, notifications, etc.). BlaBlaCar également propose des applis natives. Cependant, développer nativement deux apps exige deux équipes distinctes, ce qui n'est pas réaliste pour une petite équipe. **Les solutions cross-plateformes modernes n'existaient pas au lancement de ces compagnies**, mais aujourd'hui des frameworks comme **React Native** ou **Flutter** offrent un compromis intéressant : un seul code mobile pour Android et iOS, avec des performances proches du natif. D'ailleurs, de nombreux "clones" ou nouveaux acteurs du covoiturage optent pour des technos cross-plateformes pour démarrer plus vite ¹⁰ ¹¹ . Par exemple, un guide de développement de carpooling note que les stacks courantes incluent *React Native* pour le mobile cross-plateforme, couplé à un backend Node.js et parfois Firebase pour les fonctionnalités en temps réel ¹¹ . Ceci montre qu'une startup de covoiturage peut tout à fait réussir en partant sur du cross-platform pour le mobile et un backend Node, sans immédiatement égaler l'infrastructure complexe d'un Uber.

- **Bases de données et scalabilité des données** : Initialement, une base de données relationnelle unique peut suffire. Uber a tenu avec une seule instance PostgreSQL pendant son enfance ² . À l'augmentation du trafic, Uber est passé à une approche plus segmentée des données : ils ont migré certaines données vers des sharded MySQL (solution interne "Schemaless"), et utilisé des bases NoSQL (Riak, Cassandra) pour les besoins à très forte échelle ou en temps réel, tout en mettant en place du caching Redis ¹² ¹³ . BlaBlaCar, en grandissant, a dû migrer d'un hébergement on-premise vers le cloud (GCP) et adopter Kubernetes pour gérer plus efficacement la charge et la croissance internationale ⁹ ¹⁴ . **La leçon pour Ko'Ride** est qu'une base SQL standard (PostgreSQL ou MySQL) peut gérer les données de l'application MVP, mais il faudra l'architecturer pour évoluer (par exemple en prévoyant des index géospatiaux pour rechercher les trajets par localisation, en pouvant partitionner les données par région si besoin, etc.). On retient aussi que l'ajout de **caches en mémoire** (ex: Redis) et éventuellement de bases NoSQL spécifiques peut se faire plus tard pour optimiser les performances lorsque l'audience explose. En somme, commencer simple, puis adopter des solutions de stockage distribuées au fur et à mesure que ça devient nécessaire, est un chemin éprouvé par ces entreprises.

En résumé, les grands noms du secteur nous enseignent de **privilégier la simplicité et la rapidité de développement au départ**, tout en gardant en tête une évolution possible vers une architecture plus modulaire. Il faut adopter des outils connus et maîtrisables par l'équipe actuelle et les recrues locales

potentielles, et concevoir le système de façon modulaire pour pouvoir le faire évoluer (scale-out) quand le volume d'utilisateurs l'imposera.

Frontend web : Angular pour le site web

Étant donné que l'équipe a une expertise en Angular, il est logique de l'exploiter pour le front web de Ko'Ride. Angular est un framework front-end robuste et complet, bien adapté pour construire une application web riche en fonctionnalités. Il permettra de développer une interface web pour Ko'Ride où les utilisateurs pourront rechercher des covoiturages, gérer leur profil, publier des trajets, etc. Les avantages d'Angular dans ce contexte sont :

- **Productivité et structuration** : Angular fournit une structure MVC/modulaire qui aide un développeur junior à organiser le code en composantes, services, modules, etc. Ceci correspond aux bonnes pratiques de maintenabilité. De plus, comme Jean-Charles et Yann le maîtrisent déjà, le développement sera plus rapide qu'avec une technologie nouvelle pour eux.
- **Écosystème et outils** : Angular dispose d'un écosystème mature (CLI pour scaffolder le projet, nombreux composants UI, form binding, etc.). Des bibliothèques existantes pourront être utilisées pour intégrer par exemple Google Maps dans la page (via Angular Google Maps), gérer l'authentification via JWT, etc., ce qui accélère le développement.
- **TypeScript** : Angular utilise TypeScript, ce qui apporte une sécurité de typage qui aide à réduire les bugs, tout en restant dans l'univers JavaScript familier. Cette homogénéité TS/JS pourra être un atout si le backend utilise aussi TypeScript (nous y reviendrons).
- **Performance côté client** : Angular, bien que plus lourd que des frameworks comme React en terme de bundle initial, offre de bonnes performances runtime et des outils de lazy-loading qui seront utiles pour charger par exemple la carte seulement quand nécessaire. Pour le marché indonésien, il faudra veiller à optimiser l'application pour les connexions mobiles parfois limitées (par ex, en gérant un PWA *Progressive Web App* pour que le site soit consultable hors-ligne ou avec faible réseau, et en minimisant le poids des bundles).
- **Communauté et recrutement** : Angular est utilisé dans le monde entier, y compris en Asie du Sud-Est, même si React y est également très populaire. On peut s'attendre à pouvoir recruter des développeurs Angular ou à former relativement vite des développeurs frontaux indonésiens sur Angular, étant donné que c'est un framework bien documenté et largement enseigné.

En complément, il serait judicieux d'implémenter dès le départ un design **responsive** et mobile-friendly sur l'application Angular, afin que le site web puisse servir de solution de secours pour les utilisateurs n'installant pas l'application mobile. Une PWA (Progressive Web App) pourrait être envisagée – Angular le supporte – mais étant donné la nature du covoiturage (notifications push, géolocalisation continue, intégration profonde GPS), **une application mobile native ou cross-platform sera nécessaire** pour l'expérience optimale des utilisateurs.

Application mobile : privilégier une solution cross-plateforme

Le marché indonésien est très axé mobile – Android y domine largement, avec également une base iOS non négligeable parmi les professionnels et classes moyennes ¹⁵. Pour toucher le plus grand nombre, Ko'Ride devra proposer une application mobile sur **Android et iOS**. Étant donné la petite taille de l'équipe et l'absence d'expertise mobile native en interne, le choix d'un **framework cross-platform**

s'impose pour le MVP. Les deux principales options modernes sont **React Native** et **Flutter**, auxquelles on peut ajouter Ionic (Angular) comme outsider. Analysons ces options :

- **React Native (RN)** : RN permet de développer en JavaScript/TypeScript, en utilisant le framework React. Avantage pour l'équipe : le langage serait le même que celui d'Angular (TypeScript), et Jean-Charles en fullstack JS pourrait se former plus aisément à React Native. La communauté RN est grande, de nombreux modules existent (pour Google Maps, pour les notifications, etc.), ce qui accélère le développement. L'inconvénient est qu'il faudrait apprendre la philosophie React (basée sur les composants, hooks, etc.), différente de celle d'Angular. Néanmoins, RN est éprouvé : de nombreuses apps populaires l'utilisent. Il offre des performances proches du natif grâce au rendu natif des composants UI. RN pourrait donc être un bon choix si l'équipe se sent prête à apprendre React.
- **Flutter** : Flutter, développé par Google, utilise le langage Dart. C'est un framework cross-plateforme qui a l'avantage de **fournir des performances quasiment natives** (le code Dart est compilé en natif) et d'offrir une très grande rapidité de développement grâce au *hot-reload* et à un riche ensemble de widgets préconçus. Flutter est particulièrement apprécié pour la cohérence de l'UI entre Android et iOS – on code une seule interface qui rendra de façon identique sur les deux plateformes. Pour Ko'Ride, Flutter présente plusieurs atouts : rapide pour construire des interfaces complexes (par ex. affichage de la carte, animations fluides), bon support des fonctionnalités comme la géolocalisation (plugins Google Maps, etc.), et **croissance de son écosystème en Indonésie**. En effet, Flutter gagne en popularité dans la région, et il peut être plus facile de recruter de jeunes développeurs locaux connaissant Flutter/Dart, car la courbe d'apprentissage est relativement douce et Google fait la promotion de Flutter dans les communautés dev. Le principal frein est que personne dans l'équipe actuelle ne connaît Dart/Flutter, il faudra donc apprendre un nouveau framework. Cependant, comme toute l'équipe part de zéro en mobile, apprendre Flutter ou RN représente un effort équivalent, avec peut-être l'avantage d'une documentation Flutter très didactique et d'une base de code unifiée.
- **Ionic / Angular + Capacitor** : Cette option consisterait à réutiliser les compétences Angular pour créer une application hybride. Ionic permet de développer une application mobile en Angular (ou React/Vue) et de la déployer dans une WebView native via Capacitor ou Cordova, avec accès aux API natives. L'avantage est la réutilisation maximale d'Angular (donc un terrain connu pour l'équipe) et une seule base de code web+mobile. Cependant, les inconvénients sont notables pour un cas comme le covoiturage : les performances sont généralement inférieures (application web encapsulée, moins fluide qu'une vraie native ou Flutter app, surtout pour les animations ou la carte), et les limitations d'accès aux fonctionnalités avancées (GPS en tâche de fond, notifications très réactives) peuvent poser problème ou nécessiter des plugins parfois moins robustes. Ionic pourrait servir pour un prototype rapide, mais si Ko'Ride vise une qualité d'application du niveau de Grab ou BlaBlaCar, il vaut mieux s'orienter vers RN ou Flutter qui donnent un rendu **100% natif** au final.

Compte tenu de ces points, **notre recommandation penche pour Flutter** comme solution mobile cross-plateforme. Flutter offrirait une vitesse de développement élevée et une expérience utilisateur de qualité native sur les deux plateformes phares ¹⁶ ¹⁷ . De plus, choisir Flutter serait cohérent avec le fait qu'Angular est déjà utilisé sur le web, puisque Google maintient les deux technologies (les concepts de composant/widget ont quelques similitudes dans l'approche déclarative UI). Flutter étant de plus en plus adopté, l'équipe pourra éventuellement recruter des développeurs Flutter en Indonésie assez facilement, où la communauté grandit ¹⁸ ¹⁹ . Néanmoins, si l'équipe se sent plus à l'aise avec l'écosystème JavaScript, **React Native reste un excellent choix**, soutenu par une vaste communauté, et

pourrait tirer parti du savoir-faire TypeScript existant. Dans les deux cas, le bénéfice est d'éviter d'avoir deux bases de code (Android + iOS) à maintenir.

En termes de fonctionnalités mobiles clés à implémenter, on aura besoin de : - **Géolocalisation & Cartographie** : Intégration de Google Maps ou Mapbox pour afficher les trajets, repérer les conducteurs/passagers sur la carte, calculer des itinéraires. Flutter comme RN ont des bibliothèques prêtes (ex: `google_maps_flutter` pour Flutter, ou `react-native-maps` pour RN). - **Notifications push** : Indispensables pour alerter de nouveaux messages, confirmations de covoiturage, etc. On utilisera Firebase Cloud Messaging (FCM) pour Android et l'APNS pour iOS – souvent géré via Firebase pour simplifier (Flutter et RN ont des plugins pour FCM) ²⁰. - **Stockage local & mode offline** : Prévoir un cache local (par ex. SQLite embarqué ou Hive base de données locale sous Flutter) pour conserver l'historique des trajets ou messages en mode déconnecté. - **Performances et optimisation** : Prendre en compte que beaucoup d'utilisateurs auront des smartphones Android d'entrée/milieu de gamme. Flutter a l'avantage d'un très bon rendu même sur des appareils modestes, et RN nécessite d'optimiser le travail sur le fil JavaScript pour ne pas bloquer l'UI. Dans tous les cas, il faudra tester sur des terminaux variés.

En conclusion, **développer l'app Ko'Ride en cross-platform** permettra de livrer rapidement sur Android et iOS sans doubler les coûts. Le choix Flutter est recommandé pour sa rapidité de développement et son UI homogène, mais React Native serait un choix tout à fait valable si l'équipe préfère rester dans l'écosystème JS. Dans tous les cas, la web app Angular continuera d'exister en parallèle pour les utilisateurs sur desktop ou pour référencement web, et possiblement comme PWA de secours.

Backend centralisé : API unifiée pour web et mobile

Le backend de Ko'Ride sera le cœur névralgique, exposant des API consommées par l'application mobile et le site web. Ce backend devra gérer les comptes utilisateurs, la création et recherche de trajets, la réservation, les paiements, la messagerie interne, etc. Il devra également supporter des fonctionnalités temps-réel (ex: mise à jour de la position du conducteur, notifications instantanées). Voici nos recommandations pour la stack backend, en tenant compte des préférences de l'équipe (Java Spring Boot vs Node.js/NestJS) et des exigences du projet :

- **Langage et framework backend** : Étant donné l'expérience de David (backend) et de Jean-Charles (fullstack) qui semblent orientés web, une option naturelle est **Node.js avec un framework TypeScript**. En particulier, **NestJS** pourrait être un excellent choix, car il s'inspire fortement de l'architecture Angular (décorateurs, injection de dépendances, structure modulaire) ce qui faciliterait la prise en main pour l'équipe ²¹. NestJS tourne sur Node.js, bénéficiant de son modèle événementiel et non-bloquant, idéal pour les applications temps réel avec de nombreuses connexions simultanées. Cela correspond bien aux besoins d'une appli de covoiturage où l'on peut avoir de multiples utilisateurs rafraîchissant la liste des trajets ou envoyant des messages. D'ailleurs, Uber a montré qu'un service critique écrit en Node.js pouvait gérer un grand nombre de connexions concurrentes grâce à l'asynchronisme ⁶. En outre, un backend Node/TypeScript permettrait une **cohérence de langage** front-back (tout en TS), ce qui simplifie le développement pour une petite équipe (les développeurs peuvent plus facilement naviguer entre le front Angular et le back NestJS).

En alternative, **Java Spring Boot** est un framework puissant, utilisé en entreprise, avec lequel on peut tout à fait construire une API REST robuste. Spring Boot offre du *batteries-included* (sécurité, accès BD, etc.) et de hautes performances multi-thread. Cependant, pour une équipe junior, Spring Boot pourrait avoir une courbe d'apprentissage plus raide (configuration, concepts comme les Beans, etc.), surtout

sans mentor expérimenté en Java. Node.js/NestJS serait plus léger en termes de charge cognitive et pourrait accélérer le prototypage. De plus, du point de vue **recrutement en Indonésie**, il y a de nombreux développeurs Java dans le secteur (grâce à Android et aux entreprises J2EE), mais il y a également toute une nouvelle génération à l'aise avec JavaScript/Node. Node.js étant très populaire pour les startups et projets web, il pourrait être plus facile de trouver des profils Node/JS junior motivés pour rejoindre Ko'Ride.

En somme, nous recommandons **NestJS (Node.js + TypeScript)** pour le backend MVP. Cela n'exclut pas d'éventuellement intégrer Spring Boot ou un autre langage plus tard pour des microservices spécifiques si un besoin particulier se présente (par exemple un module de calcul d'itinéraire très poussé qui pourrait bénéficier de Java/Go), mais démarrer en Node/Nest offre le meilleur compromis rapidité/commodité.

- **API : REST ou GraphQL ?** Pour servir à la fois le front web Angular et l'appli mobile, il faudra concevoir des APIs. Le choix entre REST et GraphQL dépend des besoins et des compétences. **REST** est le choix le plus simple et universel : l'équipe peut rapidement créer des contrôleurs RESTful dans NestJS ou Spring Boot exposant des endpoints JSON. C'est largement compris par tous les développeurs (plus facile à confier à des recrues juniors) et bien supporté (ex: Angular a HttpClient pour consommer REST). **GraphQL** offre plus de flexibilité côté client (on peut demander exactement les champs voulus, éviter de multiples requêtes imbriquées), ce qui peut optimiser les échanges réseau pour le mobile. Des entreprises comme Facebook ou Yelp utilisent GraphQL, et on voit apparaître des API GraphQL dans des services de mobilité, mais ce n'est pas (à notre connaissance) la norme chez Uber/Grab pour les apps mobiles grand public – Uber privilégie plutôt gRPC ou des appels REST optimisés en interne. Pour Ko'Ride, **on suggère de démarrer en REST**, plus simple à mettre en place. NestJS permettrait éventuellement de supporter GraphQL plus tard via un module dédié, si on constate un bénéfice (par ex, pour unifier en un appel les données de profil + trajets + messages qu'autrement plusieurs endpoints REST auraient fournis). Mais au stade MVP, la clarté d'une API REST bien conçue sera un atout. Chaque ressource principale (utilisateurs, trajets, réservations, paiements, messages) aura son set d'endpoints CRUD et d'actions spécifiques. Il conviendra de sécuriser ces endpoints (authentification via token JWT, voir section sécurité).

- **Gestion du temps réel (notifications, géolocalisation live)** : Un point crucial pour Ko'Ride sera la gestion des fonctionnalités *live* : suivre la position d'un conducteur sur la carte en temps réel, recevoir instantanément un message ou une confirmation de réservation. Deux approches peuvent se combiner :

- **WebSockets** : Grâce à Node.js, on peut utiliser Socket.io ou la librairie WebSocket native pour pousser des informations en temps réel du serveur vers les clients ²² . Par exemple, dès qu'un conducteur met à jour sa position ou qu'un nouveau message de chat est posté, le backend envoie l'update aux clients concernés via un canal WebSocket plutôt que d'attendre que ceux-ci fassent une requête. NestJS intègre bien Socket.io. C'est approprié pour un suivi de position avec mise à jour toutes les X secondes, et pour le chat instantané.

- **Firebase (BaaS)** : Une alternative ou complément consiste à déléguer certains aspects temps réel à des services managés. Par exemple, **Firebase Realtime Database** ou **Firestore** peut servir à diffuser la position en direct ou les messages de chat – le client Flutter/RN peut écouter les changements de la base en temps réel sans gérer l'infrastructure serveur WebSocket. De même, **Firebase Cloud Messaging** (FCM) sera utilisé de toute façon pour les notifications push système (car sur mobile, on doit passer par FCM/APNS) ²⁰ . L'équipe pourrait choisir d'utiliser Firebase pour accélérer le développement du MVP (surtout si les compétences backend sont limitées). Par

exemple, stocker les conversations de chat dans Firestore simplifierait énormément la mise en place de la messagerie (Firebase s'occupe de la synchro temps réel, de la scalabilité, etc.) ²³ ²² . Néanmoins, s'appuyer exclusivement sur Firebase peut limiter la liberté de développement sur le long terme et engendrer des coûts variables. Un compromis peut être intéressant : **utiliser Firebase pour les notifications push et éventuellement le chat en temps réel**, tout en gardant le backend principal (Node) pour la logique métier, la persistance des trajets et réservations, etc. Cela rejoint ce que proposent certains tutoriels de carpooling : une architecture hybride où *“notre propre serveur gère la logique cœur, et Firebase sert pour des flux en temps réel spécifiques”* ²³ .

- **Base de données principale** : Nous préconisons d'utiliser une base de données **relationnelle SQL** comme socle, en particulier **PostgreSQL** (ou MySQL en alternative). Ces SGBD sont open-source, fiables et bien maîtrisés dans l'industrie, y compris pour des applications de covoiturage. Ils garantissent la cohérence des transactions (important pour, par exemple, ne pas sur-réserver une place dans une voiture ou pour tracer les paiements). PostgreSQL notamment offre des extensions géospatiales (PostGIS) très utiles pour interroger les données de localisation – on pourra par exemple effectuer une requête pour trouver tous les trajets dont le point de départ est dans un rayon de X km d'un utilisateur. BlaBlaCar, par exemple, a opéré longtemps sur du SQL (et utilise encore certainement du SQL pour une partie de ses données structurées) avant d'ajouter d'autres outils ² . Uber a commencé sur PostgreSQL avant de migrer vers MySQL pour certaines données ² , ce qui prouve que ces technologies peuvent encaisser une croissance significative.

Concrètement, on pourra modéliser dans PostgreSQL les entités de Ko'Ride : Utilisateurs, Profils, Véhicules, Trajets (propositions de covoiturage avec ville de départ/arrivée, horaires, places dispo, etc.), Réservations (quand un passager réserve une place sur un trajet), Paiements, Avis/Notes, Messages (bien que pour le chat on puisse aussi utiliser une solution NoSQL temps réel comme évoqué). Une base SQL bien normalisée fonctionne bien pour ces données liées. Si le trafic augmente, on pourra verticalement monter en puissance le serveur PostgreSQL ou passer à un cluster, et éventuellement réfléchir à du sharding (par région géographique par ex) ou à passer à un service cloud managé (Cloud SQL, Amazon RDS) qui facilite la montée en charge. Uber a montré qu'on peut aller assez loin avec une base relationnelle unique avant de devoir la segmenter ¹ ² .

- **Caches et autres bases spécialisés** : Pour améliorer les performances, il serait judicieux d'intégrer un système de cache dès que l'usage monte. Par exemple, un cache en mémoire **Redis** peut stocker les résultats de requêtes fréquentes (liste des trajets populaires, configuration de l'appli, etc.) et gérer des choses comme les sessions utilisateur ou les jetons JWT invalidés, avec une latence très faible ¹³ . Redis peut aussi servir de **File d'attente** (il a une fonctionnalité pub/sub) pour propager des événements entre microservices plus tard. À plus long terme, si certaines fonctionnalités nécessitent un stockage différent : par ex., l'historique des positions GPS ou les journaux d'activité pourraient être mieux gérés dans une base NoSQL (Cassandra, MongoDB) pour absorber un volume massif d'écritures. **Cependant, pour le MVP de Ko'Ride, ceci n'est pas indispensable.** Une bonne optimisation SQL + un cache suffiront. L'architecture doit simplement garder ces possibilités d'extension en tête.
- **Intégrations tierces** : Une application de covoiturage ne vit pas en vase clos – il faudra consommer plusieurs API externes, qui influencent parfois le choix technologique:
- **API de cartographie et de géolocalisation** : L'utilisation de **Google Maps API** est pratiquement incontournable pour obtenir les coordonnées GPS des adresses saisies, tracer les itinéraires routiers et calculer éventuellement les distances/temps estimés ²⁴ ²⁵ . Cela implique d'intégrer

le SDK Google Maps côté mobile (via plugins Flutter/RN) et d'appeler l'API Directions/Geocoding côté backend pour calculer les itinéraires optimaux ou le point de rendez-vous central. On pourrait aussi évaluer **Mapbox** (open source) pour réduire les coûts, mais Google Maps offre la fiabilité et la connaissance locale (cartes indonésiennes à jour).

- **Services de paiement** : Ko'Ride devra intégrer un système de paiement en ligne pour faciliter la transaction entre passager et conducteur (à moins de partir sur un modèle entièrement cash, ce qui est peu pratique). Pour un MVP, intégrer un agrégateur comme **Stripe** ou **PayPal** peut fonctionner ²⁴, mais en Indonésie il faudra sans doute prendre en charge des solutions locales (portefeuilles électroniques du type GoPay, OVO, DANA, ou transferts bancaires locales). Une option est de passer par un prestataire indonésien (tels que Midtrans, DOKU) qui offre des API unifiées pour plusieurs méthodes de paiement locales. Techniquement, cela signifie que le backend devra communiquer de manière sécurisée avec ces API (via REST/HTTPS), gérer les webhooks de confirmation de paiement, etc. Spring Boot comme NestJS disposent de bibliothèques pour consommer des services externes facilement (HTTP client, etc.). **La sécurité** sera primordiale sur cette partie – il faudra bien stocker les clés API, éventuellement passer par un serveur proxy si les réglementations l'imposent.
- **Envoi de SMS ou emails** : Pour la vérification de numéro de téléphone (étape souvent nécessaire pour instaurer la confiance, comme le font Uber/Grab), on peut utiliser un service comme **Twilio** pour envoyer des SMS OTP. Là encore une simple requête API depuis le backend déclenchera l'envoi. On peut aussi envisager d'envoyer des emails de notification (confirmation de réservation, reçu de paiement) via un service comme SendGrid. Ces choix n'affectent pas fortement la stack, si ce n'est qu'il est plus facile de les utiliser dans un langage où de nombreux SDK existent (heureusement, Twilio/SendGrid fournissent des SDK en JS, Java, etc. donc Node ou Java pourront les utiliser).
- **Authentification sociale** : Optionnellement, permettre aux utilisateurs de se connecter via Google/Facebook peut être un plus. Là, utiliser Firebase Authentication pourrait simplifier la mise en œuvre (il gère les OAuth Google/FB, etc.), ou on peut utiliser les SDK spécifiques (Passport.js sur Node, ou Spring Social en Java par ex). Ce choix peut être fait en fonction du temps disponible, mais n'est pas indispensable au MVP.

En synthèse, pour le backend Ko'Ride, nous préconisons une **API RESTful construite avec NestJS (Node.js & TypeScript)**, connectée à une base **PostgreSQL** principale, et enrichie de caches (**Redis** à terme). Cette API gèrera la logique métier centrale. Elle s'appuiera sur les **services cloud** appropriés pour les fonctionnalités transverses : envoi de notifications push (FCM), SMS (Twilio), paiement sécurisé, et utilisation de l'API Google Maps pour tout ce qui est géographique. Cette stack backend est cohérente avec les recommandations générales pour une app de covoiturage : un article de référence suggère Node.js/Express ou Python/Django comme langages backend pour ce type de projet, couplés à PostgreSQL ou MongoDB selon les besoins ²⁶ ²⁷ – ici nous choisissons Node/SQL pour les raisons exposées. L'accent doit être mis sur une **architecture modulaire** du code (par domaines métier) afin de faciliter l'évolution future.

Infrastructure, déploiement et évolutivité

Au-delà des choix de langage, il faut concevoir l'architecture de manière à pouvoir passer du stade prototype à une production à grande échelle progressivement, sans refaire entièrement la stack. Voici une feuille de route architecturale pour Ko'Ride :

- **MVP – architecture simple et économique** : Pour le prototype fonctionnel et les premiers mois de mise en service, on peut déployer l'application de façon très simple. Par exemple, héberger le backend sur un serveur cloud unique (une VM ou un container dans le cloud). Un choix pragmatique serait d'utiliser un PaaS (*Platform as a Service*) type **Heroku** ou **Railway** pour

déployer l'API Node rapidement sans gérer l'infrastructure, ou un container Docker sur une VM Amazon EC2/Google Compute Engine. Le coût de départ est faible (quelques dizaines d'euros par mois maximum). La base de données PostgreSQL peut être initialement hébergée sur un service managé (Amazon RDS, Google Cloud SQL) pour gagner en fiabilité (sauvegardes automatiques, etc.) sans effort d'administration – ces services ont des niveaux gratuits ou à bas coût suffisants pour démarrer. L'application Angular peut être hébergée statiquement (sur Firebase Hosting, Vercel, Netlify ou même un bucket S3 derrière CloudFront) pour être servie aux utilisateurs web. L'appli mobile Flutter/RN sera distribuée via les stores (Google Play et Apple App Store). À ce stade, **tous les composants pourraient être monolithiques** : un seul service backend gère tout, connecté à une seule base de données. Cette simplicité réduit les risques de bugs inter-services et les coûts. On s'assure tout de même dès le départ de séparer les *environnements* (base de données de test vs base de prod, etc.), et de mettre en place une intégration continue basique (par ex. GitHub Actions pour lancer les tests et déployer automatiquement sur le PaaS).

- **Montée en charge progressive** : Si l'application gagne en utilisateurs (par exemple, commence à avoir des milliers d'utilisateurs actifs chaque jour), il faudra s'assurer qu'elle tient la charge. Le backend Node.js peut être facilement mis à l'échelle horizontalement : on pourra exécuter **plusieurs instances** Node en parallèle (sur plusieurs conteneurs ou machines) derrière un **load balancer**. Les services cloud comme Heroku le font automatiquement (dynos), ou on peut utiliser Docker/Kubernetes sur AWS/GCP pour orchestrer plusieurs instances. L'architecture REST stateless facilite cela (toutes les instances peuvent servir indifféremment les requêtes). Pour que cela fonctionne, il faudra que les sessions utilisateur soient soit sans état (idempotentes via tokens JWT stockés côté client) soit partagées via un stockage central (ex: Redis pour stocker des sessions si nécessaire). La base de données pourra être dimensionnée plus fortement (passer sur un plan supérieur de la base managée). On introduira sans doute un **système de cache Redis** à ce stade pour absorber les lectures fréquentes sans solliciter constamment PostgreSQL, et possiblement mettre en place un CDN pour les contenus statiques (images de profil, fichiers) en les hébergeant sur un stockage objet type Amazon S3 ou Google Cloud Storage.
- **Sécurité et fiabilité en production** : En grandissant, des aspects comme la sécurité (protection contre les injections, XSS, sécurité des appels API, chiffrer les communications SSL – possiblement via un proxy NGINX en entrée), la supervision (logs, monitoring) deviennent cruciaux. Il faudra utiliser un service de monitoring (Datadog, NewRelic, ou l'open-source Prometheus/Grafana) pour suivre les performances, un traçage des erreurs (Sentry par exemple) pour repérer les bugs en production. L'infrastructure devra également être redondante : avoir **plusieurs zones** géographiques si on couvre toute l'Indonésie, afin de réduire la latence pour les utilisateurs de différentes îles par exemple. Les grands acteurs comme Uber et Grab ont des déploiements multi-régions très sophistiqués (Uber mentionne un déploiement actif-actif sur plusieurs data centers avec bascule automatique en cas de panne ²⁸), mais à notre échelle on peut s'appuyer sur les régions cloud (par exemple déployer sur un datacenter en Asie du Sud-Est, comme Jakarta ou Singapour, pour être proche des utilisateurs). BlaBlaCar, lorsqu'ils ont migré sur Google Cloud, ont justement fait ce choix pour pouvoir tester facilement sur de nouveaux marchés à l'échelle sans investissement lourd en serveurs ²⁹.
- **Évolution vers des microservices modulaires** : À terme, si Ko'Ride explose en popularité sur l'archipel indonésien, l'architecture devra évoluer pour soutenir une base d'utilisateurs massive et de nouvelles fonctionnalités. On pourra alors appliquer une transformation progressive en **microservices**. L'idée serait d'identifier des domaines du système qu'on peut isoler en services indépendants – par exemple, un service dédié aux notifications, un service dédié au moteur de recommandation/matching de trajets, un service de paiement, etc. Chacun de ces microservices pourrait être développé/déployé séparément, éventuellement avec des technos différentes si

justifié (c'est ce qu'ont fait les géants : Uber a des microservices en Go pour le temps réel, en Python pour du machine learning, etc. ³⁰). La communication entre microservices se ferait via des API REST internes ou via des messages (Kafka, RabbitMQ) pour découpler davantage. **Cependant, soulignons qu'il n'est ni nécessaire ni souhaitable de se précipiter vers les microservices trop tôt.** Il faudra d'abord avoir atteint une complexité et une taille d'équipe justifiant ce découpage ⁴. En attendant, on peut s'inspirer de principes "hexagonaux" ou modulaires dans le code monolithique pour préparer le terrain – par exemple en structurant le repo NestJS en modules par sous-domaine (module Utilisateur, module Trajet, module Paiement...), chacun avec son propre service, modèle de données, de façon à pouvoir les extraire plus aisément plus tard. BlaBlaCar indique par exemple utiliser l'architecture hexagonale dans ses codebases pour maintenir de la flexibilité ³¹. Ainsi, quand le moment viendra, extraire un module du monolithe vers un microservice indépendant sera plus aisé, car ses dépendances seront claires et limitées.

- **Scalabilité du temps réel** : Un enjeu particulier en évoluant sera la scalabilité des fonctionnalités temps réel (tracking de localisation, dispatch). À faible échelle, un serveur Node avec Socket.io peut gérer cela, mais à très grande échelle (dizaines de milliers de connexions simultanées), il faudra envisager des solutions comme des **serveurs de WebSockets dédiés** (par ex, déployer une clusterisation de Socket.io avec un store partagé, ou utiliser des services managés comme Pusher, PubNub si cela devient trop complexe en interne). Uber, par exemple, a développé des systèmes maison (Ringpop, etc.) pour la haute dispo en temps réel ³², et utilise Kafka massivement pour distribuer les événements. Ko'Ride n'en sera pas là au début, mais l'utilisation d'outils cloud modernes (comme Google Pub/Sub ou AWS SNS/SQS) peut faciliter la transition vers une architecture événementielle quand nécessaire, sans avoir à tout réécrire.

En résumé, l'architecture proposée est **progressive** : on commence avec un déploiement simple, centralisé (monolithe Node.js + PostgreSQL) qui couvre tous les besoins du MVP. Cette base doit être développée en suivant de bonnes pratiques de séparation des préoccupations pour éviter un monolithe spaghetti. À mesure que l'application grandit, on **scale horizontalement** ce monolithe (plus d'instances, cache, CDN) puis on **découpe en services** seulement quand le volume et l'organisation de l'équipe l'exigent. Cette approche itérative a fait ses preuves – "presque toutes les success stories de microservices ont débuté par un monolithe qu'on a ensuite découpé" ³³. Ko'Ride pourra donc s'inspirer de cette philosophie, en gardant en tête les objectifs de performance et de coût : utiliser le cloud pour éviter l'investissement initial (on paye à l'usage), monitorer les dépenses (le choix de technologies open-source évite les licences, et une stack légère comme Node consomme peu de ressources au départ), et optimiser progressivement.

Conclusion : Stack recommandée pour Ko'Ride

En synthèse, voici la **stack technologique recommandée** pour développer Ko'Ride dans le contexte donné :

- **Frontend Web** : Angular (TypeScript) – pour une application web riche, en profitant de l'expertise de l'équipe. L'UI sera conçue par Léo (UX/UI) en collaboration avec les devs, en veillant à l'ergonomie pour le public indonésien (langue, simplicité, confiance). Angular assurera une base solide pour le code front, et permettra éventuellement de réutiliser des morceaux dans une PWA.
- **Application Mobile** : Framework cross-plateforme Flutter (Dart) – pour cibler Android et iOS avec un seul code. Flutter offre une rapidité de développement et des performances quasi natives, ainsi qu'une cohérence de design sur les plateformes. Alternativement, React Native

(JavaScript/TypeScript) est un choix valide si l'équipe préfère, car il s'intègre bien avec un backend Node et bénéficie d'un écosystème large. Dans les deux cas, on aura les fonctionnalités mobiles cruciales (GPS, notifications) gérées via plugins.

- **Backend/API** : Node.js avec le framework NestJS (TypeScript). Cela fournira une API REST centralisée, suivant une architecture MVC modulaire. NestJS apporte une structure inspirée d'Angular qui sera familière, et tourne sur Node qui est performant en I/O et adapté aux besoins temps réel. On pourra définir des endpoints REST sécurisés (JWT auth) pour toutes les fonctionnalités. Le backend pourra utiliser Socket.io pour le temps réel, et s'appuyer sur Firebase FCM pour pousser les notifications mobiles.
- **Base de Données** : PostgreSQL comme base de données principale (relationnelle). Elle stockera de manière fiable les informations structurées (utilisateurs, trajets, réservations, paiements, avis...). PostgreSQL offre consistance et supporte les requêtes géospatiales, utiles pour le covoiturage. On prévoit la possibilité d'optimiser avec des index géographiques, et de recourir à un **cache Redis** pour accélérer les lectures fréquentes et gérer les sessions si nécessaire.
- **Infrastructure Cloud** : Déploiement sur un fournisseur cloud majeur avec présence en Asie du Sud-Est (par exemple **AWS** ou **Google Cloud**). On utilisera des services managés pour gagner du temps : instance EC2 ou service Kubernetes pour le backend, base de données managée (Amazon RDS ou Cloud SQL) pour PostgreSQL, Cloud Storage (S3 bucket) pour stocker les photos de profil ou documents, CDN pour servir le contenu statique du site web rapidement aux utilisateurs en Indonésie. BlaBlaCar, par exemple, a migré sur Google Cloud pour accompagner sa croissance mondiale ⁹, ce qui montre l'intérêt d'une infrastructure cloud scalable. Ko'Ride n'aura qu'à "monter le volume" sur ces services en cas de pic de trafic, sans changer de stack.
- **Intégrations externes** :
 - **Maps/Géolocalisation** : API Google Maps pour la géocodage (traduire adresses en coordonnées), calcul d'itinéraire et affichage cartographique. SDK Google Maps intégré côté front (web et mobile).
 - **Paieement** : Intégration d'un service type Stripe pour le MVP (paiement par carte) et préparation pour intégrer des wallets locaux. L'objectif est un paiement sans friction dans l'app, sécurisée (conforme PCI DSS).
 - **Notifications Push & SMS** : Firebase Cloud Messaging pour notifications push sur mobile ²⁰. Service SMS (Twilio ou équivalent local) pour vérification téléphone et alertes si nécessaire.
 - **Auth et Sécurité** : Gestion JWT pour l'authentification via le backend. Possibilité d'utiliser Firebase Auth ou Auth0 si on veut accélérer l'implémentation de l'inscription/connexion de façon sécurisée, mais un module NestJS JWT couplé à PostgreSQL (pour stocker les empreintes de mots de passe, etc.) peut suffire. Chiffrement SSL end-to-end via HTTPS (un certificat TLS sur le nom de domaine de l'API, géré par un proxy Nginx ou le load balancer du cloud).

Cette stack est **cohérente avec les pratiques modernes** de développement d'applications de covoiturage. Elle privilégie la productivité (Angular/NestJS/Flutter permettent un développement rapide avec une base de code maintenable) tout en s'alignant sur ce qu'on retrouve dans les stacks recommandées pour ce type d'application (par exemple, un blog spécialisé liste Flutter, Node.js, PostgreSQL, Firebase comme un combo efficace pour les apps de covoiturage ¹⁰ ¹¹). Surtout, elle est **adaptée à l'équipe junior** : utilisation de technologies connues (JavaScript/TypeScript), grand écosystème de support (communautés Angular, Node, Flutter très actives), et possibilité d'attirer des développeurs locaux qui maîtrisent ces outils.

Enfin, l'architecture proposée est **réaliste et progressive** : elle permet de sortir un MVP fonctionnel rapidement sans sacrifier l'évolutivité. À mesure que Ko'Ride grandira, l'équipe pourra s'appuyer sur les solides fondations posées (modularité du code, cloud scalable, bonnes pratiques de code) pour ajouter de nouvelles fonctionnalités (par ex. un algorithme de suggestion de covoiturage optimisé par machine learning, qui pourrait être développé en Python en microservice séparé si besoin à l'avenir). En suivant

l'exemple des grands du secteur mais à échelle humaine, Ko'Ride pourra évoluer d'une petite application locale à, nous l'espérons, une plateforme de covoiturage majeure en Indonésie, le tout avec une stack maîtrisable et performante.

Sources : Les recommandations ci-dessus s'appuient sur l'étude de cas d'applications similaires et sur les retours d'expérience de grandes entreprises du domaine. Par exemple, Uber a montré l'efficacité d'un démarrage simple (monolithe Python/Postgres) avant le passage aux microservices pour soutenir sa croissance ¹. Lyft utilise une stack Python/Flask en microservices avec du front en JS, illustrant qu'un stack web classique peut faire tourner une plateforme de mobilité ⁷. Des guides de développement d'applis de covoiturage confirment l'orientation vers Node.js/Express ou Django comme backend, et Flutter/React Native en front mobile, avec PostgreSQL comme base fiable ²⁶ ¹⁰. Enfin, BlaBlaCar et Grab ont misé sur le cloud et les microservices pour l'échelle, ce qui reste un objectif à long terme pour Ko'Ride une fois le produit validé ⁹ ³. En synthétisant ces bonnes pratiques internationales et en les adaptant au contexte de Ko'Ride, on obtient la stack technologique optimale pour allier **vitesse de développement maintenant et croissance sereine plus tard.** ² ⁶

¹ Up: Portable Microservices Ready for the Cloud | Uber Blog

<https://www.uber.com/blog/up-portable-microservices-ready-for-the-cloud/>

² ¹² ¹³ ²⁸ The Uber Engineering Tech Stack, Part I: The Foundation | Uber Blog

<https://www.uber.com/blog/tech-stack-part-one-foundation/>

³ How BlaBlaCar Works: Revenue Model Explained - IdeaUsher

<https://ideausher.com/blog/blabla-car-business-model-how-it-works/>

⁴ ³³ Starting With a Monolith or Microservices: How New Technology Is Changing the Conventional View

<https://blog.dreamfactory.com/starting-with-a-monolith-or-microservices-how-new-technology-is-changing-the-conventional-view>

⁵ ⁶ ³⁰ ³² The Uber Engineering Tech Stack, Part II: The Edge and Beyond | Uber Blog

<https://www.uber.com/blog/uber-tech-stack-part-two/>

⁷ What is the software stack of Lyft? - Quora

<https://www.quora.com/What-is-the-software-stack-of-Lyft>

⁸ TIA x Grab - Tech in Asia

<https://www.techinasia.com/tag/tia-x-grab>

⁹ ¹⁴ ²⁹ BlaBlaCar Case Study | Google Cloud

<https://cloud.google.com/customers/blabla-car>

¹⁰ ²⁰ ²² ²⁵ How To Develop A Carpooling App: Features, Steps and Cost

<https://emizentech.com/blog/develop-a-car-pooling-app.html>

¹¹ Carpooling App Development: Steps, Features, and Cost Breakdown | Codementor

<https://www.codementor.io/@rajdeep885527/carpooling-app-development-steps-features-and-cost-breakdown-2qw1ba5o1b>

¹⁵ ¹⁶ ¹⁷ ¹⁸ ¹⁹ ²³ Analysis of all docs.odt

<file:///file-Cm77XvCyFhHCZ5FExgtpEE>

²¹ nestjs #nodejs #java #springboot #comparison #developers - LinkedIn

https://www.linkedin.com/posts/piyushcodes6_nestjs-nodejs-java-activity-7229929778814824450-V9hW

²⁴ ²⁶ ²⁷ Building an App Like BlaBlaCar: A Comprehensive Guide

<https://www.linkedin.com/pulse/building-app-like-blabla-car-comprehensive-guide-vibidsoft-reigf>

31 Pragmatic Hexagonal Architecture - BlaBlaCar - Medium

<https://medium.com/blablaCar/pragmatic-hexagonal-architecture-f0dbc70b736d>